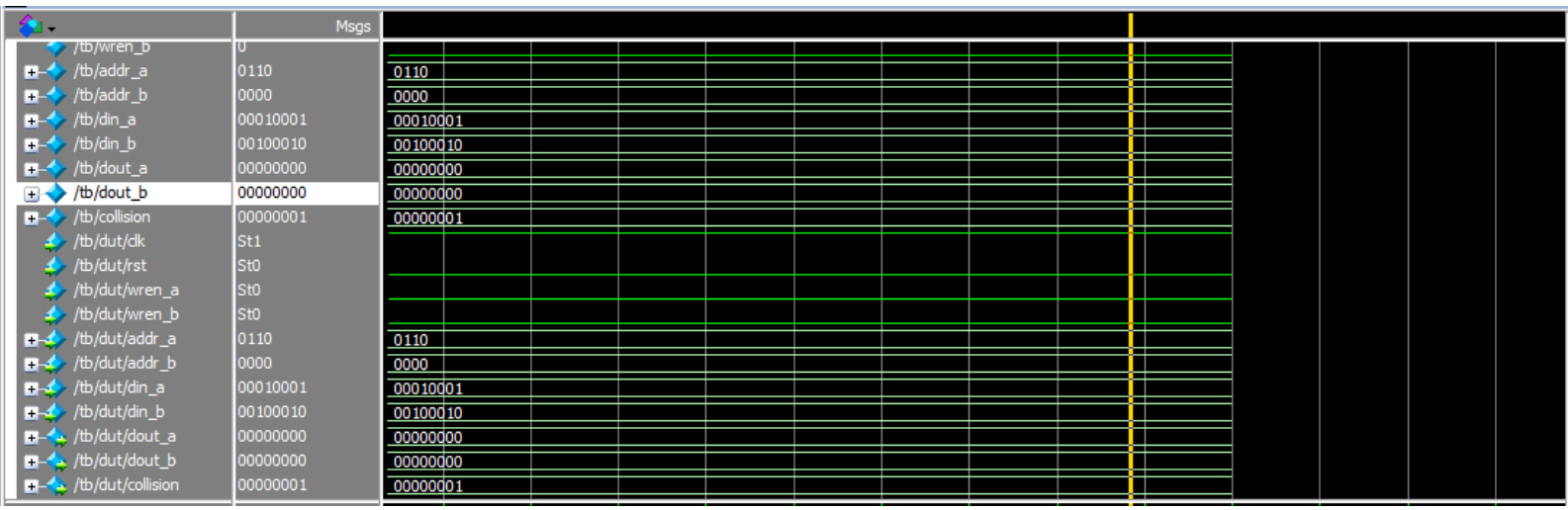
```
module dual_port_ram #(parameter ADDR_W = 4, ADDR_DEPTH = 16,
DATA_W = 8)
(
input clk,rst,
input wren_a, wren_b,
input [ADDR_W-1:0] addr_a, addr_b,
input [DATA_W-1:0] din_a, din_b,

output reg [DATA_W-1:0] dout_a, dout_b,collision
);

reg [DATA_W-1:0] mem [0:ADDR_DEPTH-1]; //memory declartion
integer i ;
always @(posedge clk)
begin
    if (rst) begin
        for (i = 0; i < ADDR_DEPTH; i = i + 1) mem[i] <= 0;
        dout_a<= 0;
        dout_b<= 0;
    end

    else if(wren_a || wren_b)
        begin
            collision = 0;
            if(addr_a == addr_b)    collision = 1;

            if (wren_a) mem[addr_a] <= din_a;
            dout_a <= mem[addr_a];

            if (wren_b) mem[addr_b] <= din_b;
            dout_b <= mem[addr_b];
        end
end
endmodule
```


Msgs
/tb/wren_b
/tb/addr_a
/tb/addr_b
/tb/din_a
/tb/din_b
/tb/dout_a
/tb/dout_b
/tb/collision
/tb/dut/clk
/tb/dut/rst
/tb/dut/wren_a
/tb/dut/wren_b
/tb/dut/addr_a
/tb/dut/addr_b
/tb/dut/din_a
/tb/dut/din_b
/tb/dut/dout_a
/tb/dut/dout_b
/tb/dut/collision
0
0110
0000
00010001
00100010
00000000
00000000
00000001
St1
St0
St0
St0
0110
0000
00010001
00100010
00000000
00000000
00000001
0110
0000
00010001
00100010
00000000
00000000
00000001

```
module fifo #(parameter DEPTH = 8,DATA_WIDTH = 8, PTR_WIDTH = $clog2(DEPTH))
(
input clk, rst_n,
input w_en, r_en,
input [DATA_WIDTH-1:0] data_in,
output reg [DATA_WIDTH-1:0] data_out,
output full, empty
);


reg [PTR_WIDTH:0] w_ptr, r_ptr;
reg [DATA_WIDTH-1:0] fifo [0:DEPTH-1];


always @(posedge clk)

begin
    if (!rst_n)
    begin
        w_ptr <= 0;
        r_ptr <= 0;
        data_out <= 0;
    end

    else if (r_en && !empty)
    begin
        data_out <= fifo[r_ptr];
        r_ptr <= r_ptr + 1;
    end

    else if (w_en && !full)
    begin
        fifo[w_ptr] <= data_in;
        w_ptr <= w_ptr + 1;
    end

end

assign full = (w_ptr[PTR_WIDTH] != r_ptr[PTR_WIDTH]) && (w_ptr[PTR_WIDTH-1:0] == r_ptr[PTR_WIDTH-1:0]);
assign almost_full = (w_ptr - r_ptr == 2);
assign empty = (w_ptr == r_ptr);
assign almost_empty = (w_ptr - r_ptr == 2) && (w_ptr != r_ptr);

endmodule
```